

Модели разработки ПО

Чтобы лучше разобраться в том, как тестирование соотносится с программированием и иными схемами проектной деятельности, для начала рассмотрим самые основы – модели разработки ПО (как часть жизненного цикла ПО). При этом сразу отметим, что разработка ПО является лишь частью жизненного цикла ПО, и здесь мы говорим именно о разработке.

Выбор модели разработки ПО серьезно влияет на процесс тестирования, определяя выбор стратегии, расписания, необходимых ресурсов и т.д.

Моделей разработки ПО много, но в общем случае классическими можно назвать **каскадную**, **v-образную**, **итерационную**, **инкрементальную**, **спиральную** и **гибкую**.

Знать и понимать модели разработки ПО необходимо затем, чтобы уже с первых дней работы понять, что происходит вокруг, что, зачем и почему Вы делаете. Чем полнее вы будете наблюдать за картиной, происходящей на проекте, тем яснее вам будет виден ваш вклад в общее дело и смысл того, чем вы занимаетесь.

Еще одна важная вещь, которую следует понимать, состоит в том, что никоим образом модель не является догмой или универсальным решением. Нет идеальной модели. Есть та, которая хуже или лучше подходит для данного проекта, данной команды, конкретных условий.

Каскадная модель (водопад)

Каскадная (водопадная) модель сейчас представляет собой, скорее, исторический интерес, т.к. в современных проектах практически не применимо. Она предусматривает однократное выполнение каждого из этапов проекта, которые, в свою очередь, строго следуют друг за другом (рисунок 1). Очень упрощённо можно сказать, что в рамках этой модели в любой момент времени станция «видна» лишь предыдущая и следующая фаза. В этой же разработке ПО необходимо «видеть весь проект» и

переходить к начальным этапам, чтобы исправить недоработки или что-то уточнить.

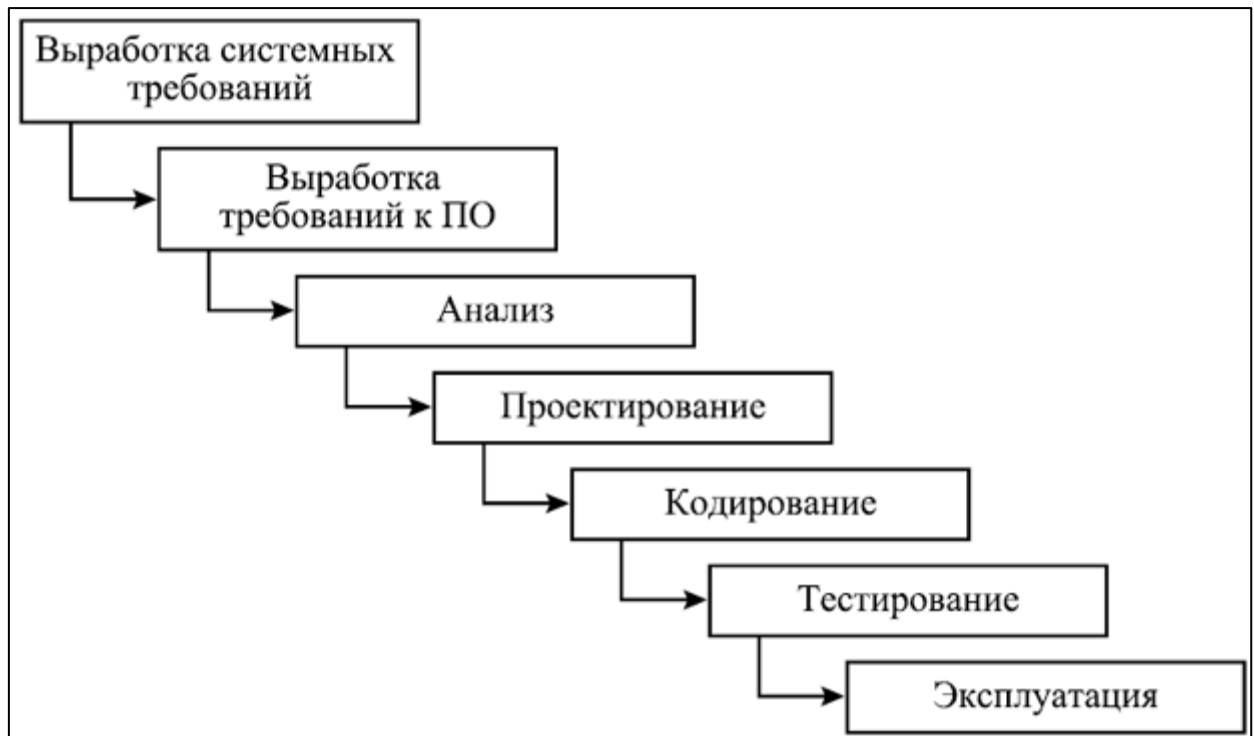


Рисунок 1 – Каскадная (водопадная) модель

Особенности каскадной модели:

- высокий уровень формализации процессов;
- большое количество документации;
- жесткая последовательность этапов жизненного цикла без возможности возврата на предыдущий этап.

Минусы:

- Водопад-проект должен постоянно иметь актуальную документацию. Обязательная актуализация проектной документации. Избыточная документация.
- Очень негибкая методология.
- У заказчика нет возможности ознакомиться с системой заранее и даже с системой «Пилотом».
- У пользователя нет возможности привыкнуть к постепенному продукту.

- Все требования должны быть представлены в начале жизненного цикла проекта.
- Возникает необходимость в жёстком управлении и регулярном контроле, иначе проект быстро выбивается из графиков.
- Отсутствует возможность рассмотреть переделку, весь проект делается за один раз.

Плюсы:

- Высокая прозрачность разработки и этапов проекта.
- Чёткая последовательность.
- Стабильность требований.
- Строгий контроль управления проектом.
- Облегчает работу по составлению плана проекта и сбору команды проекта.
- Хорошо определены процедуры контроля по качеству.

«Водоворот» или каскадная модель с промежуточным контролем

В этой модели предусмотрен промежуточный контроль за счет обратных связей. Но это преимущества и недостатки. Эта модель, как вы уже поняли, представляет собой небольшую предыдущую модификацию и относится к первой группе.

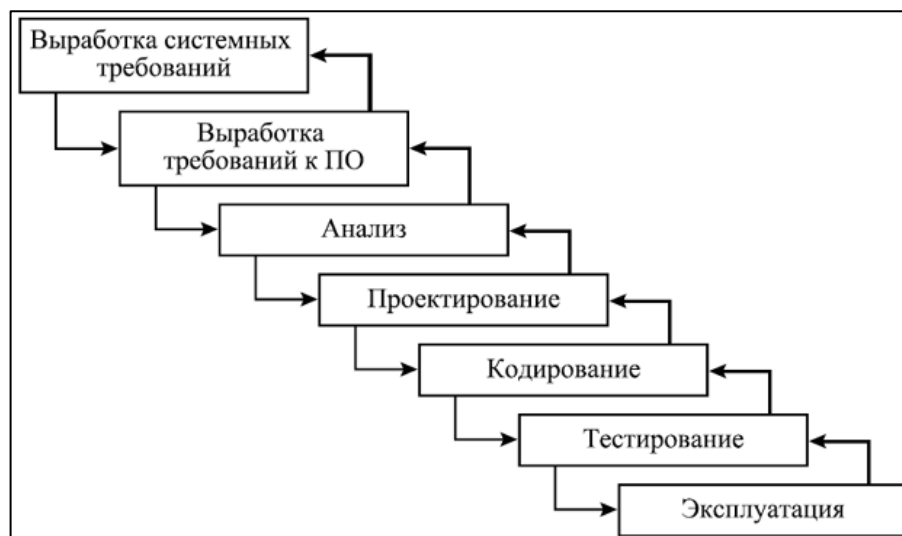


Рисунок 2 – «Водоворот» или каскадная модель с промежуточным контролем

При определении работы, в соответствии с моделью, допускающей движение только в одну сторону, обычно возникают проблемы при обнаружении недоработок и ошибок, допущенных на начальных этапах. Но еще более тяжело приходится иметь дело с изменениями окружения, в разрабатываемом ПО.

Итеративная модель

Итеративные или инкрементальные модели (известно несколько таких моделей) предполагают развитие созданной системы на наборе кусков, которые разрабатываются с помощью нескольких последовательных проходов всех работ или их частей.

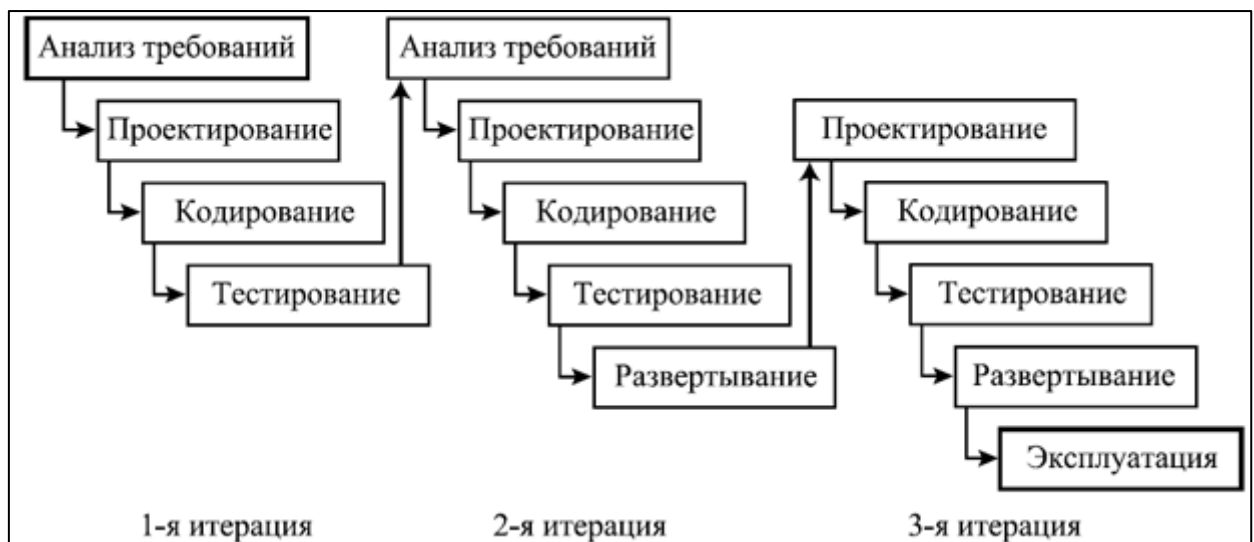


Рисунок 3 – Итеративная модель

Каскадная модель с возвращением на предыдущий шаг, при необходимости пересмотреть его результаты, становится итеративной.

Итеративный процесс предполагает, что разные виды деятельности не требуют соблюдения этапов разработки, а процедуры по необходимости иногда повторяются до тех пор, пока не будет получен желаемый результат.

При использовании итеративного алгоритма значительно сложнее оценить текущее состояние проекта и спланировать события долгосрочного развития, а также предсказать сроки и ресурсы, необходимые для обеспечения определенного качества результата.

Спиральная модель жизненного цикла программного обеспечения

Данная модель прекрасно сочетает в себе прототипирование и проектирование по стадиям, и из возникших концепций в этой модели было взято все лучшее.

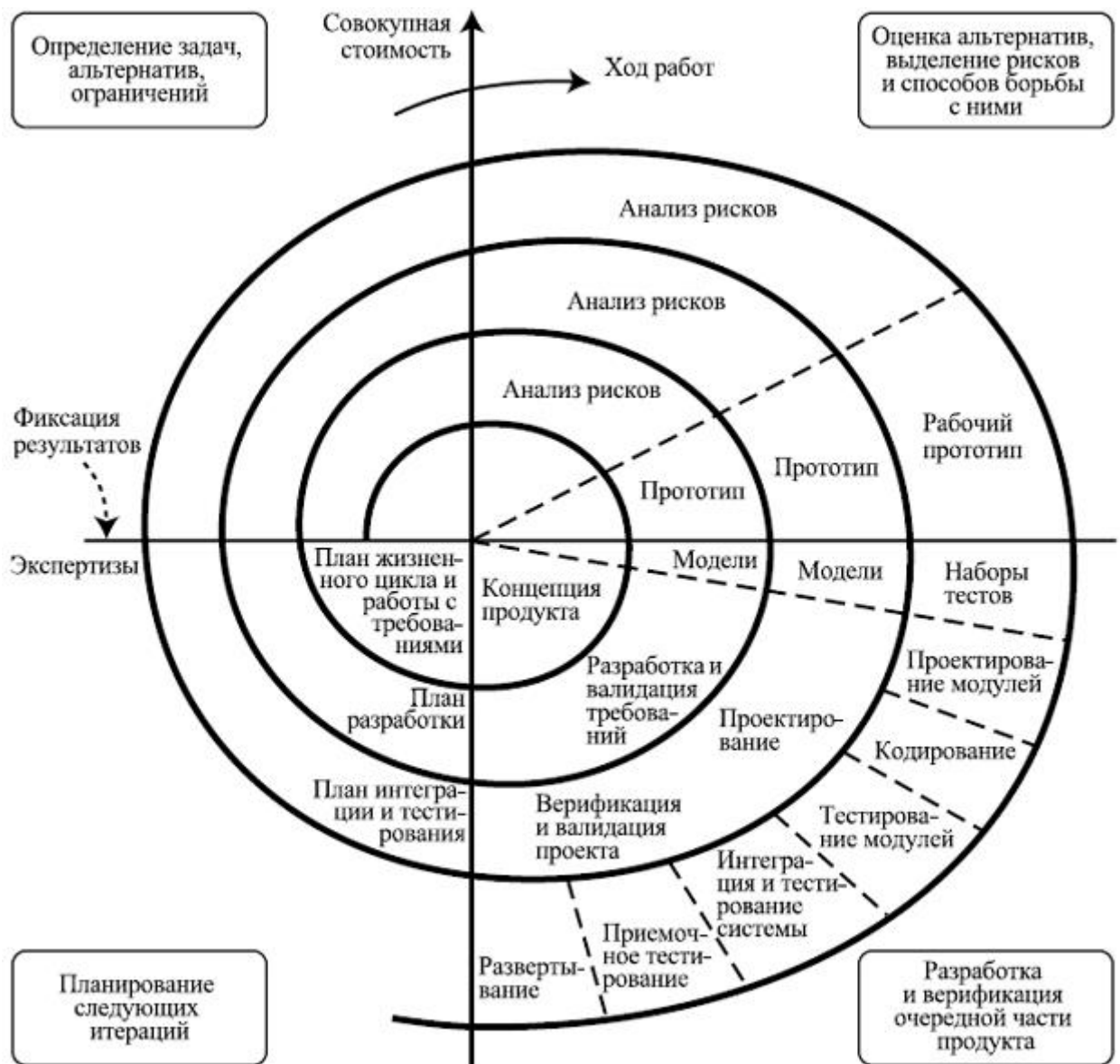


Рисунок 4 – Спиральная модель

Но в этой модели есть один существенный недостаток: невозможность регламентирования стадий выполнения.

Модель V – разработка через тестирование

Данная модель более приближена к современному методическому методу, однако все еще имеет ряд недостатков. Это одна из основных практик экстремального программирования, которая предполагает регулярное обновление продуктов во время разработки.

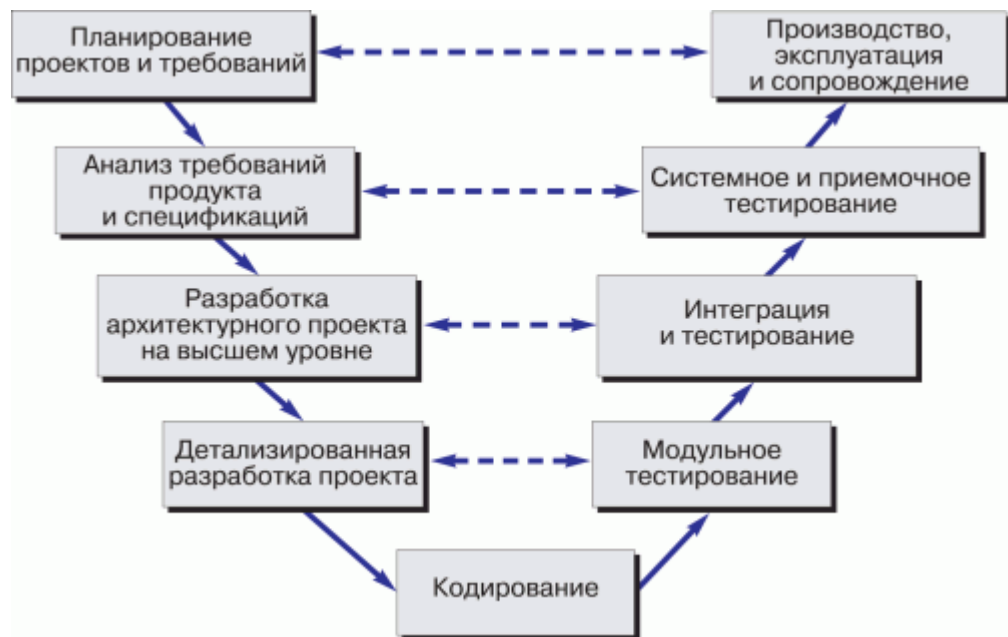


Рисунок 5 – Модель V

V-модель обеспечивает поддержку планирования и реализации проекта. В ходе проекта ставятся следующие задачи:

- **Минимизация риска:** V-образная модель, делающая проект более ясным и повышающим стандарты качества контроля проекта путем промежуточных целей и описания соответствующих результатов и ответственных лиц. Это позволяет выявлять отклонения и риски в проекте на более поздних стадиях и повышать качество управления проектами, снижая риски.

- **Повышение и гарантии качества:** V-Model – стандартизованная модель разработки, позволяющая добиться от проекта результатов желаемого качества. Промежуточные результаты могут быть подтверждены на ранних стадиях. Универсальное документирование обеспечивает читаемость, понятность и проверяемость.

- **Уменьшение общей стоимости проекта:** ресурсы для развития, производства, управления и поддержки могут быть заранее просчитаны и проконтролированы. Получаемые результаты также универсальны и легко прогнозируются. Это снижает затраты на сложные стадии и проекты.

- **Повышение качества общения между участниками проекта:** универсальное описание всех элементов и условий предполагает

взаимопонимание всех участников проекта. Таким образом, уменьшаются неточности в отношениях между пользователем, покупателем, поставщиком и разработчиком.

Модель на основе разработки прототипа

Данная модель основана на разработке прототипов и прототипировании продуктов и относится ко второй группе.

Прототипирование используется на начальных стадиях жизненного цикла программного обеспечения:

- Прояснить неясные требования (прототип пользовательского интерфейса).
- Выбрать одно из принципов концептуальных решений (происходит реализация).
- Проанализировать работоспособность проекта.

Классификация прототипов:

- **Горизонтальные прототипы** – моделирует исключительно пользовательский интерфейс, не затрагивая логику обработки и базы данных.
- **Вертикальные прототипы** – проверка архитектурных решений.
- **Одноразовые прототипы** – для быстрой разработки.
- **Эволюционные прототипы** – первое приближение эволюционной системы.

Вкратце можно выразить суть разработки моделей ПО таблицы 1.

Таблица 1 – Суть разработки моделей ПО

Модель	Преимущества	Недостатки	Тестирование
Водопадная	- У каждой стадии есть чёткий проверяемый результат. - В каждый момент времени команда выполняет один вид работы. - Хорошо работает для небольших задач.	- Полная неспособность адаптировать проект к изменениям в требованиях. - Крайне позднее создание работающего продукта.	С середины проекта.
V-образная	- У каждой стадии есть чёткий проверяемый результат. - Внимание тестированию уделяется с первой же стадии. - Хорошо работает для проектов со стабильными требованиями.	- Недостаточная гибкость и адаптируемость. - Отсутствует раннее прототипирование. - Сложность устранения проблем, пропущенных на ранних стадиях развития проекта.	На переходах между стадиями.
Итерационная инкрементальная	- Достаточно раннее прототипирование. - Простота управления итерациями. - Декомпозиция проекта на управляемые итерации.	- Недостаточная гибкость внутри итераций. - Сложность устранения проблем, пропущенных на ранних стадиях развития проекта.	- В определённые моменты итераций. - Повторное тестирование (после доработки) уже проверенного ранее.
Спиральная	- Глубокий анализ рисков. - Подходит для крупных проектов. - Достаточно раннее прототипирование.	- Высокие накладные расходы. - Сложность применения для небольших проектов. - Высокая зависимость успеха от качества анализа рисков.	
Гибкая	- Максимальное вовлечение заказчика. - Много работы с требованиями. - Тесная интеграция тестирования и разработки. - Минимизация документации.	- Сложность реализации для больших проектов. - Сложность построения стабильных процессов.	В определённые моменты итераций и в любой необходимый момент.

Основополагающие принципы Agile-манифеста

Мы следуем таким принципам:

- Наивысшим приоритетом для нас является удовлетворение клиента благодаря ранней и регулярной поставке работающего ПО.
- Изменение требований приветствуется даже на поздних стадиях разработки. Agile-процессы позволяют использовать изменения в программном обеспечении для обеспечения конкурентного преимущества заказчика.
- Работающий продукт следует выпускать как можно чаще, с периодичностью от пары недель до пары месяцев.
- На протяжении всего проекта разработчики и представители бизнеса должны ежедневно работать вместе.
- Над проектом должны работать мотивированные профессионалы. Чтобы работа была сделана, создайте условия, обеспечьте поддержку и полностью доверьтесь им.
- Непосредственное общение является наиболее практичным и эффективным способом обмена информацией как с самой командой, так и внутри команды.
- Работающий продукт – основной показатель прогресса.
- Инвесторы, разработчики и пользователи должны иметь возможность поддерживать постоянный ритм бесконечно. Agile поддерживать устойчивый темп работы.
- Постоянное внимание к техническому совершенству и качеству планирования повышает гибкость проекта.
- Простота – искусство, минимизация лишней работы – необходима крайне.
- Самые лучшие требования, архитектурные и технические решения рождаются в самоорганизующихся командах.
- Команда системы должна проанализировать возможные способы повышения эффективности и соответствующим образом скорректировать стиль своей работы.

СКРАМ

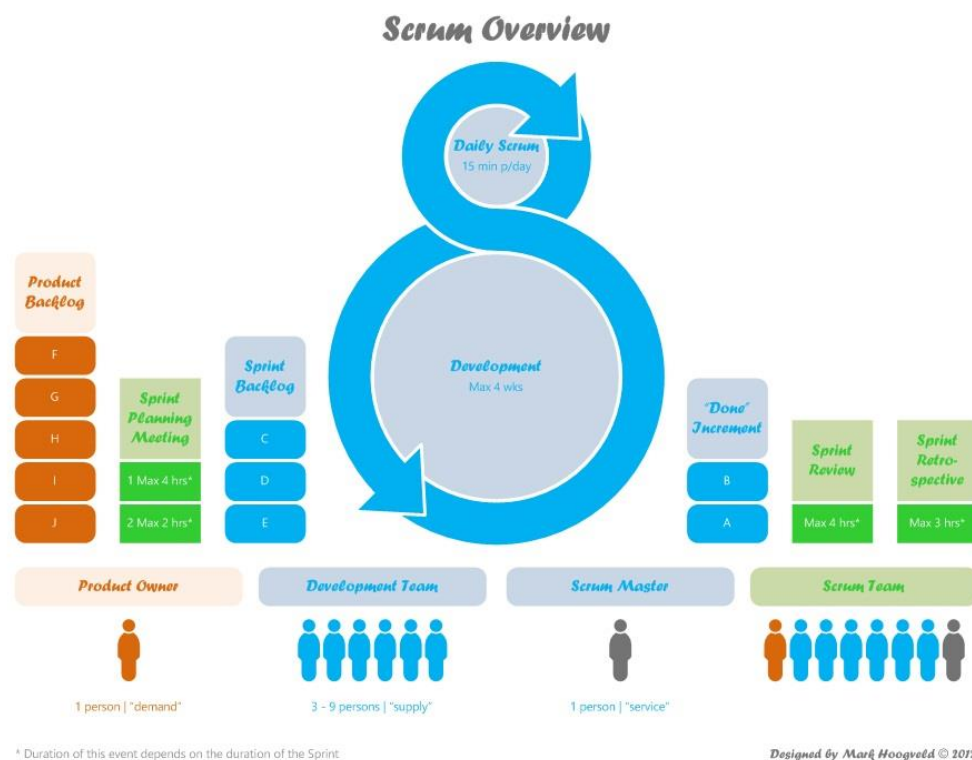


Рисунок 6 - Скрам

Сам термин Scrum можно определить так – это методология управления проектами, которая построена на принципах тайм-менеджмента. Основной ее причиной является прозрачность процесса для всех участников, причем у каждой стороны есть своя определенная роль. Суть в том, что не только команда работает над решающими задачами, но и все мы, кому интересно решение задач. Не просто поставили задачу и расслабились, а постоянно «работают» с одной командой, и эта работа не означает только постоянного контроля.

Основные термины, используемые в методологии:

- **Владелец продукта (Product Owner)** – человек, который имеет непосредственный интерес к качественному традиционному продукту, он понимает, как этот продукт должен выглядеть/работать. Этот человек не работает в команде, он работает на стороне заказчика/клиента (это может быть как другая компания, так и другой отдел), но этот человек работает с командой.

- **Скрам-мастер** – это человек, которого можно назвать руководителем проекта, хотя это не совсем так. Главное, что этот человек следует за тем, чтобы все принципы Scrum соблюдались.

- **Scrum-команда** – это команда, которая принимает все принципы Scrum и готова с ними работать.

- **Спринт** – отрезок времени, который берется для выполнения определенного (ограниченного) списка задач. Рекомендуется брать 2-4 недели (длительность определяется один раз).

- **Бэклог** – это список всех работ. Можно сказать, это ежедневный общий пользование. Создают 2 вида бэклогов: Product-бэклог и спринт-бэклог.

1. **Продукт-бэклог** – это полный список всех работ, при реализации которых мы получаем конечный продукт.

2. **Спринт-бэклог** – это список работ, который определил и согласовал с Владелцем продукцией на ближайший отчетный период (спринт). Задания в спринт-бэклоге берутся из продуктов-бэклога.

- **Планирование спринта** – это совещание, на котором присутствуют все (команда, Scrum-мастер, Владелец продукта). В течение этого срока владелец продукта определит приоритетные задачи, которые он хотел бы увидеть выполненными по объему спринта. Команда измеряет время, сколько желаемого они могут откорректировать. В итоге получается список заданий, который не может измениться в течение спринта и к завершению спринта должен быть полностью выполнен.

Жизненный цикл спринта

1. Планирование спринта.

В начале каждого спринта составляется планирование этого самого спринта. В планировании спринта участвуют заказчики, пользователи, менеджмент, владелец продукта, скрам-мастер и команда.

Планирование спринта состоит из двух последовательных митингов.

1. Планирование спринта, митинг первый.

Участники: команда, Владелец продукта, Скрам-мастер, пользователи, менеджмент.

Цель: определить цель спринта (Sprint Goal) и Sprint Backlog – функция, которая будет разработана для следующего спринта для достижения целей спринта.

Артефакт: Журнал спринта

2. Планирование спринта, митинг второй.

Участники: Скрам Мастер, команда

Цель: определить, как именно будет разрабатываться определенная функция для того, чтобы достичь цели спринта. Для каждого элемента бэклог спринта определяет список задач и измеряет их продолжительность.

Артефакт: в Sprint Backlog сложные задачи

Если в ходе спринта команда оценивает, что команда не может добиться запланированного на спринте, то Скрам Мастер, Владелец продукта и выясняют, и оценивают, как можно сократить объем работ и при этом достичь цели спринта.

2.Остановка спринта (Sprint Abnormal Termination).

Спринт может быть остановлен командой или Владелец продукта, если цель стала недостижима

После остановки спринта проводится митинг с командой, где обсуждаются причины остановки спринта. После этого начинается новый спринт: составляются его планы и начинаются работы.

3.Daily Scrum Meeting.

Этот митинг проходит каждое утро в начале дня. Он предназначен для того, чтобы все члены команды знали, кто и чем занимается в проекте. Длительность этого митинга строго ограничена и не должна превышать 15 минут. Цель митинга – поделиться информацией. Он не предназначен для решения проблем проекта. Все требующие специальных дискуссионных вопросов должны быть адаптированы к условиям митинга.

Скрам митинг проводит Скрам Мастер. Он по кругу задает вопросы каждому члену команды:

- Что сделано вчера?
- Что будет сделано сегодня?
- С какими проблемами столкнулись?

Диаграмма сгорания задач (Burndown Chart)

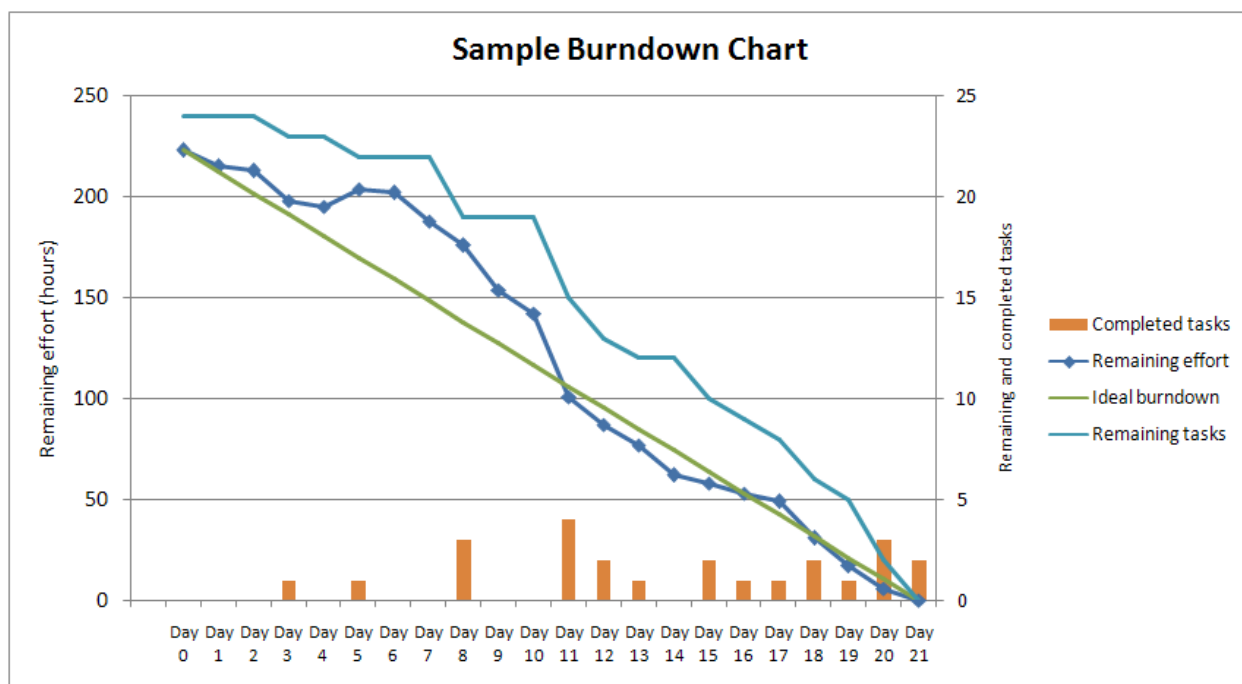


Рисунок 7 – Диаграмма сгорания задач

Диаграмма, которая показывает количество сделанной и оставшейся работы. Обновляется ежедневно с темой, чтобы в простой форме показать подвижки в работе над спринтом. График должен быть общедоступен.

Находятся разные виды диаграмм:

- **Диаграмма цилиндрической работы для спринта** – показывает, сколько уже задач сделано и сколько еще осталось сделать в современном спринте.
- **Диаграмма работ по выпуску проекта** показывает, сколько задач уже сделано и сколько еще осталось сделать до выпуска продукта (обычно строится на базе нескольких спринтов).

В конце каждой команда Спринта Скрам собирается на ретроспективу. **Цель:** просмотреть качество существующих процессов,

привлечь людей и применяемые инструменты. Команда определяет, что прошло хорошо, а что прошло не очень, а также выявляет потенциальные возможности для улучшений. Они составляют план улучшений на будущее.

Канбан

Термин «канбан» имеет дословный перевод: «Кан» означает видимый, визуальный, а «бан» – карточка или доска.

Основная задача карты «канбан» в данной системе – это уменьшение количества «выполняющейся в данный момент работы» (work in Progress).

Например, на всей производственной линии можно выделить ровно 10 карточек для дверей. Это означает, что в каждый момент времени на линии не будет больше 10 готовых дверей. Когда заказывать новые двери и сколько – это задача для того, кто их устанавливает. Только он знает свои изделия, и только он может разместить заказы на производителя дверей, но он всегда ограничен числом 10.

Этот метод Бережливого производства (Бережливое производство) был придуман в Тойоте, и сейчас многие производственные компании по всему миру его внедрили или уже внедрили.

Но это всё относится к производству, а не к разработке программного обеспечения.

А что же такое канбан–разработка, применительно к ПО, и чем она отличается от других гибких методологий, будь то SCRUM или XP?

Во-первых, нужно сразу понять, что канбан – это не конкретный процесс, а система затрат. Как, кстати, и SCRUM с XP. Это значит, что никто Вам не скажет, что и как делать по шагам.

Во-вторых, весь канбан можно описать одной простой фразой – «уменьшение выполнения выполняемой в данный момент работы».

В-третьих, канбан – это ещё более «гибкая» методология, чем SCRUM и XP. Это означает, что она не подходит для всех проектов и для всех проектов. И это также означает, что команда должна быть еще более готова к гибкой работе, чем даже команды, использующие SCRUM и XP.

Разница между канбаном и SCRUM:

- В канбане нет таймбоксов ни на что (ни на задачи, ни на спринты).
- В канбане нет фиксированного набора задач на итерацию.
- В канбане оцениваются сроки по опциональным обязательствам или вообще их нет.
- В канбане не используется velocity, используется cycle time.

Канбан-разработка отличается от SCRUM, в первую очередь, ориентацией на задачи. Если в SCRUM основная ориентация команды – это успешное выполнение спринтов (надо признать, что это так), то в канбане на первом месте задача.

Спринтов нет, команда работает над частью с самого начала и до завершения. Задачи по развертыванию выполняются тогда, когда она готова. Презентация выполненной работы – тоже. Команда не должна оценивать время выполнения задачи, потому что это имеет мало смысла и почти всегда ошибочно на начальном этапе.

Задача менеджера – это создать приоритетный пул задач, а задача команды – выбрать как можно больше задач из этого пула. Всё. Никакого контроля не нужно. Всё, что нужно от менеджера – это добавить задачи в этот пул или изменить их приоритет. Именно так он руководил проектом.

Команда для работы использует канбан-доску. Например, она может выглядеть так:

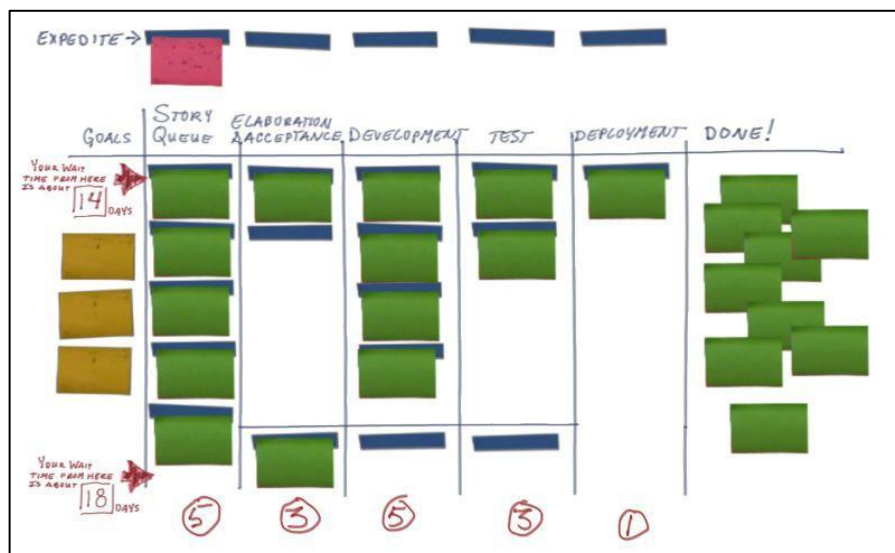


Рисунок 8 – Канбан-доска

Столбцы слева направо:

- **Цели проекта:** необязательный, но полезный столбец. Сюда можно поместить высокоуровневые цели проекта, чтобы объединить их девушку и все о них знали. Например, «Увеличить скорость работы на 20%» или «Добавить поддержку Windows 7».

- **Очередная задача:** вот задачи хранения, которые готовы к этому, чтобы начать их выполнение. Всегда для выполнения берется верхняя, наиболее приоритетная задача, и ее карточка перемещается в следующий столбец.

- **Проработка дизайна:** этот и остальные столбцы до «Закончено» могут меняться, т.к. Именно решает, какими шагами команда преодолевает проблему до состояния «Закончено». Например, в этом случае могут возникнуть задачи, разработка кода или интерфейса которых еще не решена и не обсуждается. Когда обсуждение закончено, задача переносится в следующий столбец.

- **Разработка:** тут задача посетить техпор, пока разработка фичи не завершена. После заключения она передвигается в следующий столбец. Или, если архитектура неверна или неточна, задачу можно вернуть в предыдущий столбец.

- **Тестирование:** в этом случае задача существует, пока она не тестируется. Если найдены ошибки – возвращается в Разработку. Если нет – передвигается дальше.

- **Развертывание:** во всех проектах свой развертывание. У кого-то это значит выложить новую версию продукта на сервер, а у кого-то – просто закомитировать код в репозитории.

- **Закончено:** сюда стикер попадает только тогда, когда все работы по задаче закончены полностью.

В любой работе случаются срочные задачи. Запланированные или нет, но такие, которые надо сделать прямо сейчас. Для таких можно выделить

специальное место (на картинке отмечено, как «Expedite»). В Expedite можно поместить одну срочную задачу, и команда должна начать ее выполнять немедленно и завершить как можно быстрее. Но может быть только одна такая задача! Если появляется еще одна – она должна быть добавлена в «Очередь задач».

А теперь самое важное. Видите цифры под каждым столбцом? Это число задач, которые могут быть одновременно в этих столбцах. Цифры подбираются экспериментально, но, считается, они должны зависеть от числа разработчиков в команде. Например, если вы имеете 8 программистов в команде, то в строку «Разработка» вы можете поместить цифру 4. Это значит, что одновременно программисты будут делать не более 4-х задач, значит, у них будет много причин для общения и обмена опытом. Если вы поставите туда цифру 2, то 8 программистов, занимающихся двумя задачами, могут заскучать или терять слишком много времени на обсуждениях. Если поставить 8, то каждый будет заниматься своей задачей и некоторые задачи будут задерживаться на доске надолго, а ведь главная задача канбан – это уменьшение времени прохождения задачи от начала до стадии готовности.

Никто не даст точный ответ, какими должны быть эти лимиты, но попробуйте, для начала, разделить число разработчиков на 2 и посмотреть, как это работает в вашей команде. Потом эти числа можно подогнать под вашу команду. Под «разработчиками» понимаются не только программисты, но и другие специалисты. Например, для столбца «Тестирование» разработчики – это тестеры, т.к. тестирование – это их обязанность.

Экстремальное программирование (XP)

Двенадцать основных приёмов экстремального программирования могут быть объединены в четыре группы:

1. Короткий цикл обратной связи (Fine-scale feedback):

- Разработка через тестирование (Test-driven development).

- Игра в планирование (Planning game).
- Заказчик всегда рядом (Whole team, Onsite customer).
- Парное программирование (Pair programming).

2. Непрерывный, а не пакетный процесс:

- Непрерывная интеграция (Continuous integration).
- Рефакторинг (Design improvement, Refactoring).
- Частые небольшие релизы (Small releases).

3. Понимание, разделяемое всеми:

- Простота (Simple design).
- Метафора системы (System metaphor).
- Коллективное владение кодом или выбранными шаблонами проектирования.

- Стандарт кодирования (Coding standard or Coding conventions).

4. Социальная защищенность программиста (Programmer welfare):

- 40-часовая рабочая неделя (Sustainable pace, Forty-hour week).

RATIONAL UNIFIED PROCESS (RUP)

RATIONAL UNIFIED PROCESS (RUP) – методология разработки программного обеспечения, созданная компанией Rational Software.

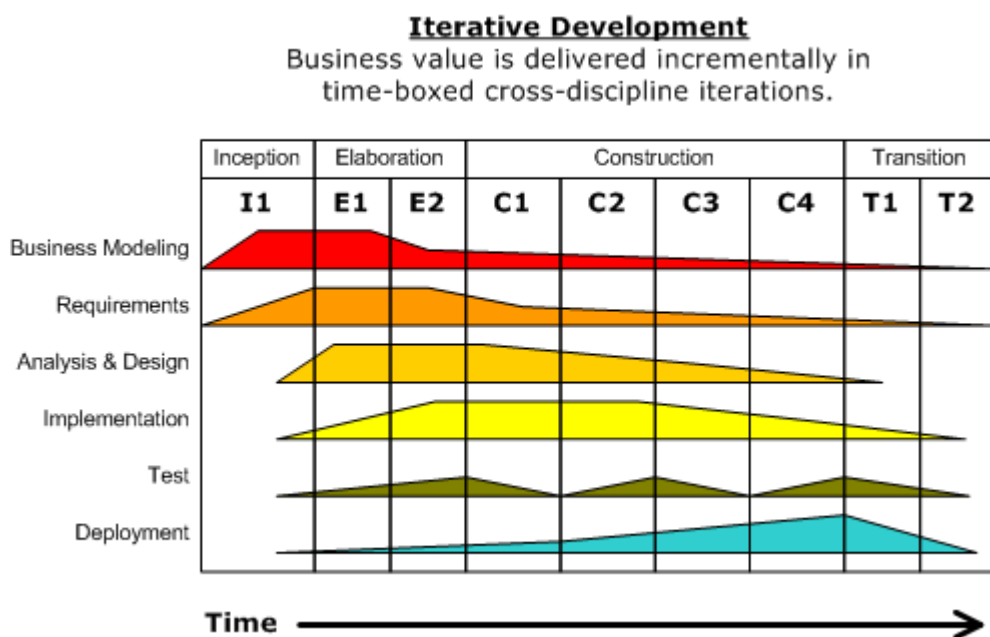


Рисунок 9 – Методология разработки ПО

В основе методологии исключения 6 основных принципов:

- Компонентная архитектура, реализуемая и тестируемая на ранних стадиях проекта.
- Работа над проектом в сплоченной команде, ключевая роль в которой принадлежит архитекторам.
- Ранняя идентификация и непрерывное устранение возможных рисков.
- Концентрация на выполнении требований заказчиков к исполняемой программе.
- Ожидаемые изменения в требованиях, проектных решениях и реализации в процессе разработки.
- Постоянное обеспечение качества на всех этапах разработки проекта.

RUP — гибкая итеративная методология, позволяющая настраивать степень формализации документации под конкретный проект, благодаря чему она одинаково хорошо подходит как для небольших быстрых задач, так и для крупных сложных систем с требованиями сертификации.